JAVA PRACTICAL PROGRAM REPORT

Longest Substring Without Repeating Characters

Program/Class Name: LongestSubstring.java

Language: Java

Topic: Scanner input, String methods (length(), charAt(), indexOf()), nested loops, character repetition detection, and string concatenation.

This report includes the aim, requirements, algorithm, UML-style diagram, complete source code, execution commands, sample console output, and conclusion.

1. Aim

To find the longest substring in which no character repeats and display that substring and its length.

2. Requirements

• Java Development Kit (JDK)
• Text editor or Java IDE
• Save the file using the public class name.

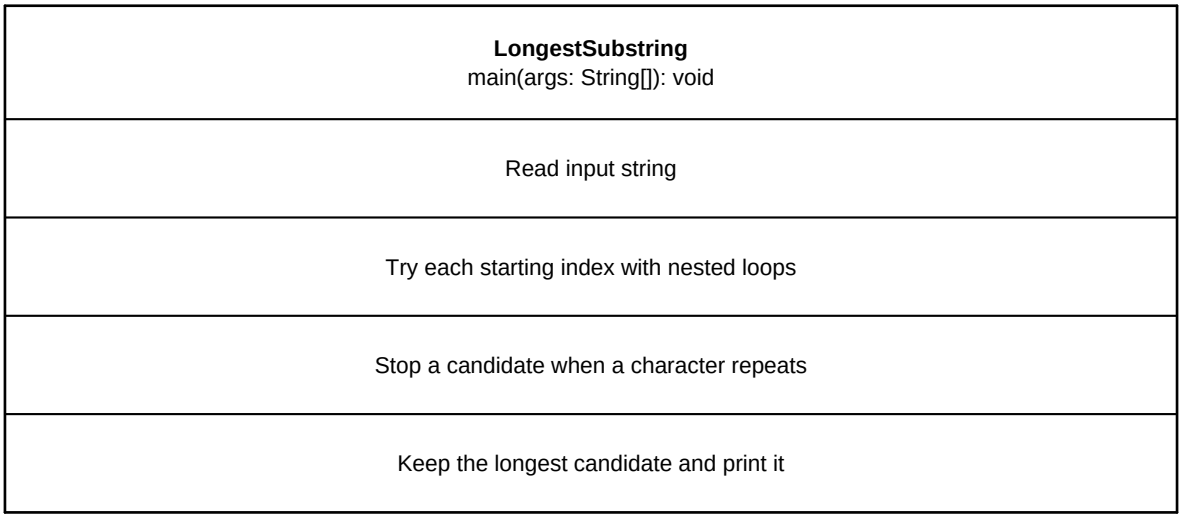
3. Concepts Used

Scanner input, String methods (length(), charAt(), indexOf()), nested loops, character repetition detection, and string concatenation.

4. Algorithm

1. Start the program and read a string from the user.
2. Initialize longest as an empty string.
3. For each starting index i, initialize current as an empty string.
4. Move through the string from index j and get the current character.
5. If current already contains that character, stop extending this substring.
6. Otherwise append the character to current and update longest if current is longer.
7. After checking all starting positions, print longest and its length.
8. Close Scanner and end.

5. UML / Class Diagram



The diagram summarizes the main class, its key fields or methods, and the program flow.

6. Source Code

```
import java.util.Scanner;

public class LongestSubstring {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        String longest = "";

        for (int i = 0; i < str.length(); i++) {

            String current = "";

            for (int j = i; j < str.length(); j++) {

                char ch = str.charAt(j);

                if (current.indexOf(ch) != -1) {
                    break;
                }

                current = current + ch;

                if (current.length() > longest.length()) {
                    longest = current;
                }
            }
        }

        System.out.println("Longest Substring: " + longest);
        System.out.println("Length: " + longest.length());

        sc.close();
    }
}
```

7. Compilation and Execution

```
javac LongestSubstring.java
java LongestSubstring
```

Sample scenario: abcabcbb.

8. Console Output

The code was compiled and run in the available Java environment. Interactive inputs are shown beside prompts for readability.

```
Enter a string: abcabcbb
Enter a string: Longest Substring: abc
Length: 3
```

9. Result / Conclusion

The program checks candidate substrings from each starting position and stops a candidate when it encounters a repeated character. For the sample input abcabcbb, the first longest substring found is abc, with length 3.